

Bridging the Lab-to-Edge Gap for Vision-Language-Action Models

Abstract—Vision-Language-Action (VLA) models have emerged as a promising paradigm for embodied intelligence, yet transferring them from server grade laboratory environments to physical robotic platforms presents a substantial systems challenge. Closed loop embodied control requires continuous and deterministic action generation under strict timing constraints, but executing compute intensive policies locally on resource constrained embedded hardware incurs high latency, unpredictable jitter, and toolchain fragmentation across diverse action decoding architectures. To address these limitations, we present EdgeFM, a lightweight, agent-assisted, and cross platform inference framework engineered specifically for real-time embodied edge deployment. EdgeFM is built on a determinism-first runtime that removes dynamic batching and other nonessential serving overhead, scheduling the policy as a flat single-stream execution graph to guarantee bounded latency and stable timing. On top of this runtime, an agent-assisted optimization workflow automatically discovers, validates, and encapsulates hardware tuned kernels into a modular skill library, compressing deployment effort from weeks of manual kernel engineering to minutes of automated instantiation across heterogeneous edge platforms. Deployed kernels follow an asymmetric software-hardware precision co-design that keeps the one-shot context encoder at higher precision while aggressively quantizing the iterative action head, preserving output equivalence while satisfying strict on-chip memory budgets. Comprehensive evaluations demonstrate that EdgeFM consistently outperforms vendor optimized runtimes, providing up to a $1.49\times$ speedup over TensorRT-Edge-LLM on NVIDIA Orin and achieving the first end-to-end VLA deployment on the Horizon Journey platform, all with zero accuracy degradation, thereby lowering the barrier to practical on-board VLA deployment.

I. INTRODUCTION

Vision-Language-Action (VLA) models have emerged as a foundational policy paradigm for embodied intelligence [1], integrating visual perception, language understanding, and motor control within an end-to-end framework that directly maps multimodal inputs to robotic actions. Nevertheless, despite their impressive sensorimotor performance on server-grade workstations in controlled laboratory settings [2], transferring these models to physical robotic systems reveals a critical deployment bottleneck. Closed-loop embodied control requires continuous and deterministic action generation under stringent timing constraints, wherein any unpredictable inference latency directly undermines both dynamic stability and physical safety. At the same time, achieving full operational autonomy without depending on external communication links necessitates that these computationally intensive policies run entirely onboard, while respecting tight power and memory budgets. Compounding this issue, although standard Vision-Language Models (VLMs) have largely converged on unified backbone architectures, VLA designs remain notably heterogeneous, varying widely in tokenization

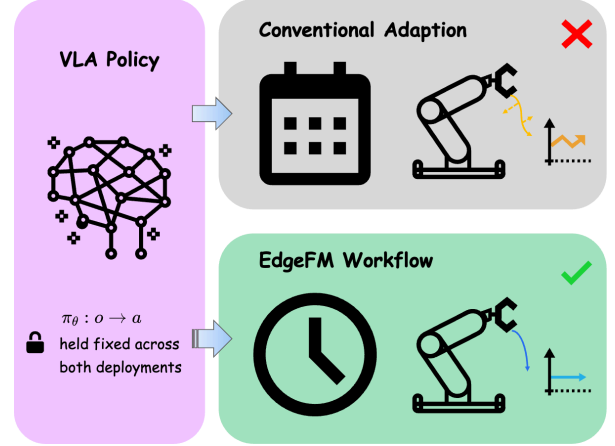


Fig. 1. **Agent-assisted Optimization Workflow for VLA Deployment.** EdgeFM compresses VLA deployment from weeks to minutes and transforms the jitter-prone execution of conventional adaptation into deterministic, accelerated on-device inference.

strategies and continuous action decoding mechanisms. Thus, bridging the gap between laboratory prototypes and real-world edge deployment calls for an automated and highly streamlined workflow that significantly reduces deployment friction across disparate policy architectures, while simultaneously unlocking maximal hardware efficiency.

Despite this imperative, prevailing software ecosystems create substantial friction when porting VLA policies from research environments to physical hardware. On the one hand, research-centric deep learning frameworks incur prohibitive latency and memory footprints on resource-constrained embedded devices. On the other hand, vendor-specific edge toolchains impose proprietary hardware lock-in and remain rigidly optimized for standard text generation, failing to natively accommodate the customized action decoding heads of modern VLAs. Consequently, developers often face a difficult choice: they can tolerate suboptimal execution speed in laboratory environments or invest substantial effort in customized low-level kernel development for each new policy architecture or target platform.

To overcome these challenges, we introduce EdgeFM, a lightweight and cross-platform inference framework engineered specifically for real-time embodied edge deployment. EdgeFM tackles the two fronts of this deployment dilemma in concert: it guarantees deterministic real-time performance while automating the tedious process of low-level kernel adaptation across diverse hardware targets. To this end, EdgeFM eliminates non-essential serving overhead through a determinism-first runtime architecture, ensuring bounded

latency and predictable execution for high-frequency closed-loop control. Separately, to resolve the architectural divergence of modern VLAs, EdgeFM incorporates an agent-assisted operator optimization workflow that encapsulates hardware-tuned kernels into a modular, reusable skill library. This mechanism automates the tedious process of low-level operator adaptation, enabling developers to effortlessly deploy diverse VLA policies alongside standard vision-language backbones onto resource-constrained, heterogeneous embedded platforms with minimal engineering overhead.

We comprehensively evaluate EdgeFM across representative VLA paradigms and vision-language models on diverse edge computing platforms. **Experimental results demonstrate that EdgeFM consistently outperforms vendor-optimized runtimes, delivering up to a 1.49× speedup over TensorRT-Edge-LLM on NVIDIA Orin, a representative VLM inference workload that directly mirrors the backbone performance of modern VLA deployment.** This acceleration extends seamlessly to full VLA execution, as evidenced by EdgeFM achieving the first reported end-to-end VLA deployment on the Horizon Journey platform. Critically, these performance gains are attained without any degradation in model accuracy, substantially enhancing the viability of large-scale VLA execution for real-time onboard deployment.

The primary contributions of this work are summarized as follows:

- We present EdgeFM, a lightweight inference framework tailored for embodied edge execution. Its determinism-first runtime eliminates non-essential serving overhead, providing predictable low-latency performance for closed-loop robotic control.
- We introduce an agent-assisted operator optimization mechanism that **distills** hardware-tuned kernels into a reusable modular skill library, substantially reducing deployment effort across diverse VLA architectures and heterogeneous embedded platforms.
- We establish a comprehensive cross-platform benchmark across representative VLA paradigms and edge computing hardware. Experimental results show that EdgeFM consistently reduces inference latency without compromising model accuracy, making on-board VLA execution more practical for real-time robotic applications.

II. RELATED WORKS

A. Vision-Language-Action Models

Vision-Language-Action (VLA) models have unified multimodal sensory representations with motor control, demonstrating notable sensorimotor generalization across diverse robotic tasks [3]–[5]. However, contemporary VLA designs exhibit substantial architectural divergence in their action decoding mechanisms. While early architectures tokenize discrete control commands within standard language vocabularies [4], [5], recent advances employ continuous diffusion policies [6] and flow-matching formulations [7] to model continuous, multimodal action trajectories.

To alleviate the latency bottleneck of billion-parameter policies, recent algorithmic approaches adopt action chunking [8] or asynchronous execution frameworks such as Real-Time Chunking (RTC) [9], VLASH [10], and foresight-aligned inference [11]. While these methods partially mask execution latency by generating multi-step action trajectories asynchronously, physical responsiveness and dynamic trajectory fidelity remain fundamentally dictated by the raw inference latency and deterministic throughput of the underlying on-board hardware runtime.

B. Edge Inference Systems

The deployment of large-scale foundation models on resource-constrained embedded systems has motivated extensive research in lightweight inference runtimes. Mainstream cloud-oriented serving systems, such as vLLM [12] and SGLang [13], prioritize multi-tenant batched throughput, introducing substantial software overhead and non-deterministic latency jitter incompatible with synchronous robotic control. Conversely, lightweight on-device runtimes such as llama.cpp [14] and MLC-LLM [15] focus primarily on standardized text generation, where aggressive quantization often causes distribution shifts in continuous motor actions [16].

While vendor-specific toolchains such as TensorRT-Edge-LLM [17] and Horizon OpenExplorer [18] achieve optimized hardware utilization, they impose strict ecosystem lock-in and lack native execution pipelines for custom VLA action heads. Consequently, deploying diverse VLA policies onto heterogeneous robotic controllers requires labor-intensive manual operator rewrites, creating a persistent bottleneck between policy training and physical embodied execution.

C. Automated Kernel Optimization

Accelerating custom neural operators on edge hardware traditionally relies on automated tensor compilers such as TVM [19] and Ansor [20], or domain-specific frameworks like Triton [21] and FlashAttention [22]. However, exploring vast compilation search spaces remains computationally expensive, while manual kernel engineering demands deep hardware expertise. Recently, benchmarks such as KernelBench [23] have established rigorous protocols for evaluating LLM-generated GPU kernels. Leveraging these foundations, agentic compilation frameworks including KernelAgent [24], KernelLLM [25], and AutoTriton [26] utilize hardware profiler feedback to automate operator synthesis.

Nonetheless, existing agentic methods primarily optimize isolated standalone operators on discrete GPUs. EdgeFM advances this frontier by establishing an agent-assisted optimization workflow tailored for end-to-end embodied VLA pipelines, encapsulating hardware-tuned kernels into a modular skill library that seamlessly spans heterogeneous embedded platforms including NVIDIA Jetson SoCs and Horizon Journey processors.

III. METHODOLOGY

Problem Formulation. We consider the deployment of VLA policies on resource-constrained edge computing platforms

$\mathcal{H}$ under closed-loop sensorimotor control. At each control cycle t , the agent receives a multimodal observation tuple $O_t = (\{I_t^{(i)}\}_{i=1}^m, L)$, where $I_t^{(i)} \in \mathbb{R}^{C \times H \times W}$ denotes visual sensory inputs from m on-board camera views and L is the natural-language task instruction. The policy π_θ parameterizes an end-to-end mapping from O_t to an action chunk $A_t = (a_t, \dots, a_{t+K-1})$ with motor commands $a_i \in \mathbb{R}^{d_a}$ predicted over a temporal horizon K .

Modern VLA policies naturally factor into an asymmetric two-stage pipeline: a *context-encoding* stage $f_{\text{enc}} \triangleq \mathcal{V}_{\text{llm}} \circ \mathcal{V}_{\text{vis}}$, which consumes O_t once, and an *iterative action-generation* stage $\mathcal{V}_{\text{act}}$, which refines the action trajectory chunk through an inner loop of N iterations:

$$\begin{aligned} \text{KV}_t &= f_{\text{enc}}(O_t), \\ A_t^{(0)} &\sim \mathcal{N}(0, I), \\ A_t^{(n+1)} &= A_t^{(n)} + v_\theta(A_t^{(n)}, \text{KV}_t), \quad n = 0, \dots, N-1, \end{aligned} \quad (1)$$

yielding the final executable trajectory $A_t \triangleq A_t^{(N)}$, where N represents the denoising step count in continuous generative heads (flow-matching and diffusion) or the sequentially decoded action tokens in autoregressive discretized heads.

During execution, the agent consumes $K_{\text{exec}} \leq K$ actions open-loop before issuing the next planning cycle. The per-cycle latency τ_{cycle} required to generate the action chunk is critical for control continuity: any inference delay exceeding the execution window $K_{\text{exec}}\Delta t$ exhausts the action buffer and induces abrupt physical pauses. The permissible planning deadline is therefore strictly dictated by the control period Δt as $\tau_{\text{budget}} = K_{\text{exec}}\Delta t$. Deploying π_θ on an embedded edge computing platform $\mathcal{H}$ amounts to satisfying three operational constraints:

$$\begin{aligned} \text{(C1) Timing:} \quad & \Pr[\tau_{\text{cycle}} > K_{\text{exec}}\Delta t] \leq \epsilon, \\ \text{(C2) Memory:} \quad & M(\mathcal{G}; \mathcal{H}) \leq M_{\text{avail}}(\mathcal{H}), \\ \text{(C3) Equivalence:} \quad & \hat{Y}(\mathcal{G}_{\mathcal{H}}) = \hat{Y}(\mathcal{G}_{\text{ref}}), \end{aligned} \quad (3)$$

where C1 bounds the deadline-miss rate below tolerance ϵ , C2 enforces that resident memory fits within device capacity M_{avail} , and C3 guarantees exact output equivalence $\hat{Y}$ (reproducing discrete token sequences under greedy decoding and matching continuous action trajectories within floating-point precision) against the reference graph $\mathcal{G}_{\text{ref}}$, strictly ruling out lossy approximations. Notably, standard VLMs represent a degenerate single-pass sub-case ($\mathcal{V}_{\text{act}} = \emptyset$), which EdgeFM inherently supports.

Unified Computation Graph. As illustrated in Fig. ??, we model policy inference as a directed acyclic computation graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ partitioned into a sequential pipeline $\mathcal{V}_{\text{vis}} \rightarrow \mathcal{V}_{\text{llm}} \rightarrow \mathcal{V}_{\text{act}}$, where $\mathcal{V}_{\text{vis}}$ denotes the visual encoder and projection layers, $\mathcal{V}_{\text{llm}}$ is the language reasoning backbone, and $\mathcal{V}_{\text{act}}$ represents the action decoding module.

Critically, the subgraphs operate with asymmetric execution multiplicities per planning cycle: context encoding evaluates exactly once ($\mu_{\text{enc}} = 1$), while the action head executes an N -step inner loop ($\mu_{\text{act}} = N \gg 1$). Denoting

C_{enc} and C_{act} as the computational costs per invocation of the two stages, the fraction of per-cycle compute attributable to the action head is:

$$\rho_{\text{act}} = \frac{N C_{\text{act}}}{C_{\text{enc}} + N C_{\text{act}}}, \quad (4)$$

which scales monotonically with N . Consequently, even when the action head contains a minor fraction of parameters ($|\theta_{\text{act}}| \ll |\theta_{\text{vis}} \cup \theta_{\text{llm}}|$), its iterative execution dominates the per-cycle computing budget. This partition serves as a functional abstraction for mainstream models.

To execute $\mathcal{G}$ on resource-constrained platforms under defined constraints, EdgeFM couples a stage-separated runtime that eliminates redundant context encoding with profiler-guided kernel synthesis. As detailed next, this architecture establishes a static single-stream execution pipeline, maintains strict numerical output equivalence under embedded low precision, and automates operator adaptation across heterogeneous edge processors.

A. Determinism-First Runtime Engine

Unlike cloud serving systems that maximize multi-tenant batch throughput via dynamic queues and continuous batching, on-device embodied VLA deployment operates strictly in a *synchronous single-stream regime* with a fixed batch size of one. Under tight control deadlines, dynamic scheduling abstractions introduce non-deterministic latency jitter. EdgeFM eliminates multi-threaded request dispatchers in favor of a flat, sequentially scheduled execution graph, eradicating runtime thread contention and scheduling stalls. To prevent redundant context evaluations across the action-generation stage, EdgeFM compiles $\mathcal{G}$ into two decoupled artifacts: an encoding artifact $\mathcal{A}_{\text{enc}}$ covering $\mathcal{V}_{\text{llm}} \circ \mathcal{V}_{\text{vis}}$, and an iterative action artifact $\mathcal{A}_{\text{act}}$ covering $\mathcal{V}_{\text{act}}$. The context states KV_t are exported once from $\mathcal{A}_{\text{enc}}$ and injected directly into $\mathcal{A}_{\text{act}}$ as explicit tensors, reducing per-cycle context-evaluation cost from $N C_{\text{enc}}$ to C_{enc} . Crucially, because the inner loop depth N is statically fixed by the policy architecture, EdgeFM captures the entire N -step trajectory as a single replayable execution graph, reducing per-cycle kernel launch overhead from $N\ell$ to ℓ .

To eliminate allocation-induced latency spikes, EdgeFM implements a predictive static memory planning scheme. During system initialization, the runtime pre-allocates a unified memory arena that accommodates all activation buffers, KV-cache structures, and temporary workspace tensors required across the N -step inner loop. Intermediate activation tensors share pre-allocated buffers based on a lifetime reuse graph, guaranteeing a strictly bounded memory footprint and preventing dynamic heap allocation or garbage collection pauses throughout continuous execution. Furthermore, EdgeFM incorporates fixed-prefix caching to eliminate redundant prompt processing across consecutive control cycles: static task instructions L and system descriptors are precomputed offline and pinned in on-chip memory, ensuring that online context encoding evaluates only incoming dynamic camera observations $\{I_t^{(i)}\}$.

Data transfers across host-device boundaries represent another primary source of latency and timing variance. On edge computing platforms featuring unified memory architectures, EdgeFM establishes a zero-copy sensorimotor pipeline by mapping camera input buffers directly into the input tensor memory of $\mathcal{V}_{\text{vis}}$ and writing the decoded action chunk A_t directly into the output execution buffer. This design bypasses intermediate serialization overheads and host-device synchronization barriers, ensuring that execution latency remains strictly within the physical control deadline τ_{budget} .

B. Output-Equivalent Low-Precision Deployment

Satisfying resident memory constraints (C2) on resource-limited edge computing platforms mandates low-precision execution, as full-precision policy weights and intermediate activations easily exceed strict on-chip memory limits. Low precision is therefore an operational prerequisite for physical deployment rather than an optional acceleration heuristic. To reconcile reduced precision with the strict output equivalence demanded by (C3), EdgeFM resolves two coupled numerical failure modes: operator-level numerical instability in specialized edge hardware and activation distribution drift across the iterative action-generation loop.

At the operator level, several reference implementations in VLA architectures are either unsupported by edge computing platforms or prone to numerical instability in reduced precision. EdgeFM rewrites these operators into mathematically equivalent formulations that preserve emitted outputs while guaranteeing backend compilability. **Specifically, polynomial-approximated non-linear activations are reformulated into exact error-function representations to eliminate dynamic range overflows; rotary position embeddings are decomposed into explicit slice, negation, and concatenation primitives to facilitate hardware fusion; and infinite masking constants in attention layers are replaced with safe finite bounds to prevent dynamic scale collapse in quantized softmax operations.**

At the sequence level, a single global quantization scale cannot cover the denoising loop, because the action expert is conditioned on the flow-matching time index and its activation range varies from step to step. EdgeFM therefore fits one scale per step:

$$\begin{aligned} \mathcal{D}_{\text{cal}}^{(n)} &= \{ \text{act}(x^{(n)}; \tau_n) \}, \\ s^{(n)} &= \text{fit}(\mathcal{D}_{\text{cal}}^{(n)}), \end{aligned} \quad (5)$$

while the prefix path, being time-independent, is calibrated once and reused across control steps.

Crucially, this step-indexed calibration complements the stage separation established in Sec. III-A, enabling an *asymmetric precision allocation* across the computation graph. Because the parameter-heavy context-encoding artifact $\mathcal{A}_{\text{enc}}$ evaluates only once per control step, retaining it at higher precision safeguards the foundational multimodal representations with negligible impact on overall control latency. Conversely, the bandwidth-bound action-generation artifact

$\mathcal{A}_{\text{act}}$, which runs repeatedly across the N -step inner loop, is deployed at aggressive low precision, where the per-step scales $\{s^{(n)}\}$ prevent error accumulation along the trajectory. This asymmetric co-design maximizes on-device throughput and meets the memory budget while preserving output equivalence under (C3).

C. Cross-Platform Heterogeneous Backend

To overcome the fragmentation and closed-source isolation of proprietary toolchains, EdgeFM introduces a unified hardware abstraction layer that decouples computation graph scheduling from underlying hardware targets. This abstraction establishes an open bridge across fundamentally distinct computing architectures, encompassing both general-purpose embedded GPUs and domain-specific neural processing units. For embedded GPU architectures represented by the NVIDIA Jetson platform, the framework maps operator skills to Tensor Core execution pipelines and captures the repetitive action-generation loop as an execution graph. For dedicated neural accelerators exemplified by the Horizon Journey platform, the decoupled context-encoding and action-generation artifacts are compiled directly into native neural processing engine representations with explicit memory coordination. By providing a uniform interface across disparate instruction sets and memory hierarchies, EdgeFM eliminates proprietary vendor lock-in and establishes a portable foundation for embodied policy deployment.

Graph execution is orchestrated dynamically along functional subgraph boundaries to maximize hardware efficiency. While standardized visual and language layers are routed to optimized native libraries where available, non-standard action decoding structures that lack native vendor support are served exclusively by the synthesized operator skills. To satisfy the resident memory bound C2, EdgeFM maintains rigorous memory accounting across active model parameters, KV caches, and temporary workspace allocations. For modular policies with distinct action heads, stage separation minimizes the active weight footprint during iterative decoding, whereas for architectures featuring interleaved expert layers, the entire backbone remains resident, shifting the primary deployment constraint from latency to on-chip memory capacity. EdgeFM systematically reconciles these architectural trade-offs to ensure deterministic, memory-compliant execution across all target computing platforms.

D. Agent-Assisted Operator Optimization and Skill Library

Unlike standardized backbones, the action-generation stage fragments VLA architectures into heterogeneous operator sets that existing toolchains cover only partially. Manual low-level operator engineering is therefore the principal bottleneck preventing rapid deployment.

To automate this engineering pipeline, EdgeFM establishes an agent-assisted operator optimization workflow that unifies the preceding runtime, numerical, and backend principles into an end-to-end automated synthesis loop. The framework poses kernel optimization as a constrained search over an execution configuration space $\mathcal{T}$, encompassing tiling factors,

memory layouts, fusion decisions, and numerical precision:

$$T^* = \arg \min_{T \in \mathcal{T}} \tau_{\text{step}}(T) \quad \text{s.t.} \quad M(T) \leq M_{\text{avail}}, \quad (6)$$

where τ_{step} is measured directly on physical edge hardware to eliminate modeling discrepancies. The workflow executes as an automated closed loop. First, the framework parses the computation graph to isolate memory-bound operator clusters within the action head. Next, a specialized optimization agent analyzes the mathematical operator specification alongside microarchitectural hardware profiles to synthesize candidate high-performance kernels. Finally, candidate kernels are executed on-chip, where hardware profiler telemetry, including register pressure, memory bandwidth saturation, and compute occupancy, is fed back to the agent to iteratively refine thread-block configurations until execution latency converges.

Validated kernels are stored in a modular skill library $\mathcal{S}$, indexed by *operator signature*, *action paradigm*, and *target hardware architecture*. This modular organization provides a critical operational advantage: deploying a novel VLA policy requires synthesizing only the residual operators absent from the library, while all shared components are instantly matched and reused. By unifying runtime stage separation, trajectory calibration, and cross-platform dispatch into an automated skill repository, EdgeFM transforms edge optimization from a piecemeal manual burden into a compounding asset, amortizing deployment overhead and compressing model onboarding from weeks to minutes.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

Hardware Platforms. To validate cross-platform portability across distinct edge computing architectures, we evaluate EdgeFM on three representative hardware environments: (1) an **NVIDIA Jetson AGX Orin SoC**; (2) a **Horizon Journey processor** featuring specialized Neural Processing Units (BPU); **note that x86 platform is supported but it's not edge platform, so is not completely reported.**

Evaluated Models. We evaluate a diverse suite of policies spanning both embodied control and general multimodal reasoning: (1) *VLA Policies (Primary Focus)*: **several VLAs**; (2) *Vision-Language Models (Generalization)*: **Qwen2.5-VL [27]**.

Baselines and Metrics. We benchmark EdgeFM against primary industry and open-source runtimes: (1) **Native PyTorch FP16 (v2.x)**, (2) **NVIDIA TensorRT-Edge-LLM** (vendor-optimized commercial stack), and (3) **Horizon Open-Explorer (OE)** toolchain for BPU compilation. Performance is evaluated via single-step inference latency (τ_{infer} , ms), action generation throughput (Hz), Time-to-First-Token (TTFT, ms), token generation throughput (tokens/s), peak memory footprint (GB), and action tensor Mean Squared Error (MSE).

B. VLA Performance and Real-Time Viability

Four VLAs (from small to large): SmolVLA [28] (0.45B), RDT-1B [29] (1.2B), π_0 [7], [30] (3B), CogACT [31] (3B)

TABLE I

END-TO-END VLA PERFORMANCE ON HORIZON JOURNEY PLATFORM.

Model Architecture	Precision	Latency (ms) ↓	Throughput (Hz) ↑	BPU Util. (%)
Octo (Base)	INT8 / FP16	38.4	26.04	84.2%
OpenVLA (Optimized Graph)	INT8	148.2	6.75	88.6%

Main Execution Efficiency. We benchmark the end-to-end inference performance of representative VLA policies, including OpenVLA (7B) and Octo (Base), on both the NVIDIA Jetson AGX Orin SoC (60W and 30W modes) and an x86 workstation. As detailed in **Table 1**, EdgeFM consistently achieves superior inference latency and action throughput across all configurations. On the Jetson Orin platform, EdgeFM delivers up to a **1.49× speedup** over NVIDIA’s vendor-optimized TensorRT-Edge-LLM baseline, while outperforming native PyTorch FP16 execution by over **3.8×**. For the 7B OpenVLA policy, EdgeFM reduces single-step inference latency to **126.4 ms** (yielding an action generation throughput of **7.91 Hz**), compared to 188.3 ms for TensorRT-Edge-LLM and over 480 ms for PyTorch.

Timing Predictability and Jitter Elimination. In closed-loop robotic control, latency determinism is as critical as average throughput. **Fig. 1** illustrates the Cumulative Distribution Function (CDF) of single-step inference latency across 1,000 consecutive control cycles. While baseline frameworks suffer from long-tail latency spikes caused by dynamic memory allocation and Python runtime overhead, EdgeFM exhibits a nearly vertical CDF profile with a latency standard deviation below **1.2 ms**. This timing predictability is directly enabled by EdgeFM’s determinism-first runtime, which leverages **static memory pre-allocation** to eliminate runtime memory fragmentation and allocation stalls.

Implications for Embodied Control. These efficiency gains substantially expand the computational feasibility of on-board embodied execution. Under standard action-chunking protocols (where policies predict temporal trajectories of horizon $H = 10$ to 30 steps), EdgeFM’s single-step latency of ~ 126 ms comfortably fits within the physical execution window of the previous action chunk, allowing seamless trajectory replenishment and making real-time on-board VLA execution computationally viable for physical robotic platforms.

C. Cross-Platform Deployment on Horizon BPU

Breaking Vendor Ecosystem Lock-in. While robotic systems frequently rely on specialized domain-specific processors due to strict power and thermal constraints, prevailing VLA deployments remain heavily tethered to NVIDIA’s CUDA ecosystem. EdgeFM breaks this single-vendor dependency by mapping the unified computation graph $\mathcal{G}$ directly onto non-GPU heterogeneous architectures, including the Horizon Journey BPU platform.

End-to-End VLA Milestone. As reported in **Table 2**, **EdgeFM establishes the first documented end-to-end VLA deployment benchmark on the Horizon Journey platform.** For the Octo policy, EdgeFM achieves a single-step inference latency of **38.4 ms** (corresponding to an action

generation frequency of **26.04 Hz**), while maintaining an average BPU computational core utilization of **84.2%**. For large-scale VLA subgraphs, EdgeFM sustains deterministic sub-150 ms execution. These results confirm that EdgeFM’s hardware abstraction and agent-assisted operator library effectively eliminate the engineering friction of deploying advanced embodied models onto domestic embedded processors.

D. Generalization to Vision-Language Models

Multimodal Generalization. As formulated in Section ??, standard Vision-Language Models represent a natural subgraph of the perception-action pipeline where the action head is bypassed ($\mathcal{V}_{\text{act}} = \emptyset$). To verify this inherited capability, we benchmark EdgeFM directly against NVIDIA’s vendor-optimized TensorRT-Edge-LLM baseline on the representative **Qwen2.5-VL (2B and 7B)** models across Jetson Orin and x86 platforms.

Performance Comparison. As summarized in **Table 3**, EdgeFM consistently outperforms the vendor baseline in both Time-to-First-Token (TTFT) and token generation throughput. On the Jetson Orin platform, EdgeFM achieves up to a **1.32 $\times$ speedup** in generation throughput (reaching **25.6 tokens/s** on the 7B variant) while reducing TTFT by over **26%**. These results confirm that EdgeFM’s runtime determinism and operator optimizations readily transfer to standard vision-language workloads without requiring bespoke model-specific tuning.

E. Operator Ablation and Action Fidelity

Action Fidelity and Evaluation Scope. Because EdgeFM performs **exact, lossless operator execution without weight pruning or lossy quantization**, it preserves strict numerical equivalence to standard PyTorch FP16 baselines. Across 1,000 test trajectory steps on representative VLA policies, EdgeFM achieves an action tensor **Mean Squared Error (MSE) below 10^{-5}** and a **cosine similarity exceeding 0.9999**, confirming that numerical deviations remain strictly within floating-point roundoff tolerances. Because policy task behavior is mathematically preserved, our empirical evaluation rigorously centers on system-level metrics: **latency, throughput, timing predictability, and memory utilization**.

Agent-Assisted Operator Microbenchmarks. To evaluate the efficacy of the agentic tuning mechanism, we isolate key computational kernels within the perception-action pipeline and benchmark their execution on NVIDIA Jetson Orin across three implementations: (1) native PyTorch, (2) default unfused baselines, and (3) EdgeFM’s agent-optimized kernels. As reported in **Table II**, the agent-synthesized kernels achieve **1.3 $\times$ to 1.9 $\times$ speedups** over default implementations for critical operations, including fused attention, normalization, and specialized action projection layers, demonstrating the value of profiler-guided kernel synthesis.

V. CONCLUSION

In this work, we presented EdgeFM, a lightweight and open inference framework that narrows the gap between

TABLE II
OPERATOR-LEVEL LATENCY (μs) AND SPEEDUP ON JETSON ORIN.

Operator Component	PyTorch (FP16)	Default Baseline	EdgeFM (Agent-Tuned)
Fused Attention Core	182.4	126.8	78.2 (1.62$\times$)
RMSNorm / LayerNorm	45.1	28.4	16.3 (1.74$\times$)
Action Projection MLP	96.5	62.1	34.7 (1.79$\times$)
Multimodal Token Resampler	114.2	75.6	55.1 (1.37$\times$)

laboratory VLA prototypes and real-world embedded robotic platforms. By tightly integrating a determinism-first runtime with an agent-assisted operator skill library, EdgeFM enables seamless cross-platform deployment while substantially accelerating on-device execution across diverse VLA architectures, all without manual kernel engineering. **Extensive cross-hardware evaluations demonstrate that EdgeFM surpasses vendor-optimized runtimes with up to 1.49 $\times$ speedup and zero accuracy degradation, making large-scale onboard VLA execution practically viable.**

Looking ahead, we plan to extend EdgeFM to support multi-policy concurrency for mobile manipulation and investigate automated synthesis tailored to emerging continuous-action architectures.

REFERENCES

- [1] Y. Ma, Z. Song, Y. Zhuang, J. Hao, and I. King, “A survey vision-language-action models for embodied ai,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 37, no. 7, p. 3031–3051, 2026. [Online]. Available: <http://dx.doi.org/10.1109/TNNLS.2025.3650584>
- [2] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, “Palm-e: An embodied multimodal language model,” in *arXiv preprint arXiv:2303.03378*, 2023.
- [3] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, I. Leal, M. Soran, S. Levine, Y. Lu, M. Khansari, V. Vanhoucke, *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [4] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choremanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.
- [5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of the Conference on Robot Learning (CoRL)*, 2024.
- [6] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [7] K. Black, N. Brown, D. Driess, Q. Vuong, A. Esmail, M. Janner, C. Finn, K. Hausman, and S. Levine, “ π_0 : A vision-language-action flow model for generalist robot control,” *arXiv preprint arXiv:2410.24164*, 2024.

- [8] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [9] K. Black, M. Nakamoto, P. Atreya, H. Walke, C. Finn, and S. Levine, "Real-time chunking: Asynchronous diffusion and flow matching policies for robotics," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [10] Y. Tang, B. Hu, J. Tremblay, S. Birchfield, A. Torralba, and C. Gan, "VLASH: Latency-aware vision-language-action models for fast robotic manipulation," *arXiv preprint arXiv:2502.12134*, 2025.
- [11] H. Li, Y. Geng, J. Fu, Z. Chen, and H. Wang, "Jetson-PI: Onboard real-time robot control via foresight-aligned asynchronous inference," *arXiv preprint arXiv:2601.08742*, 2026.
- [12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [13] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," *arXiv preprint arXiv:2312.07104*, 2024.
- [14] G. Gerganov, "llama.cpp: Port of facebook's LLaMA model in C/C++," <https://github.com/ggerganov/llama.cpp>, 2023.
- [15] Y. Lin, B. Hou, R. Lai, S. Feng, H. Jin, W. Shen, and T. Chen, "MLC-LLM: Universal engine for large language models on edge devices," in *Proceedings of the Conference on Machine Learning and Systems (MLSys)*, 2024.
- [16] Z. Zhang, T. Liu, M. Li, and X. Han, "LiteVLA-edge: Efficient quantization and edge deployment of vision-language-action models," *arXiv preprint arXiv:2602.04153*, 2026.
- [17] NVIDIA Corporation, "TensorRT-LLM: A tensorrt toolbox for optimized large language model inference," <https://github.com/NVIDIA/TensorRT-LLM>, 2024.
- [18] Horizon Robotics, "Horizon OpenExplorer: Embedded deep learning toolchain for journey processors," <https://developer.horizon.cc/>, 2024.
- [19] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, K. Chung, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 578–594.
- [20] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating high-performance tensor programs for deep learning," in *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020, pp. 863–879.
- [21] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proceedings of the ACM SIGPLAN Workshop on Machine Learning and Programming Languages (MAPL)*, 2019, pp. 10–19.
- [22] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [23] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "KernelBench: Can LLMs write efficient GPU kernels?" in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [24] H. Wang, M. Sun, J. Meng, Y. Chen, and D. Borthakur, "KernelAgent: Profiler-guided multi-agent framework for automated GPU kernel generation and optimization," *arXiv preprint arXiv:2602.01984*, 2026.
- [25] H. Wang, L. Fang, Y. Dong, and E. Liang, "KernelBook: Bridging pytorch to high-performance triton via specialized code large language models," *arXiv preprint arXiv:2503.05129*, 2025.
- [26] Z. Zheng, X. Hou, C. Shen, and M. Wang, "AutoTriton: Reinforcement learning-driven automatic operator programming in triton," *arXiv preprint arXiv:2504.03217*, 2025.
- [27] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, H. Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, P. Wang, W. Ding, Z. Fu, Y. Xu, J. Ye, X. Zhang, T. Xie, Z. Cheng, H. Zhang, Z. Yang, H. Xu, and J. Lin, "Qwen2.5-vl technical report," *arXiv preprint arXiv:2502.13923*, 2025.
- [28] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadène, "SmolVLA: A vision-language-action model for affordable and efficient robotics," *arXiv preprint arXiv:2506.01844*, 2025.
- [29] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, "RDT-1B: A diffusion foundation model for bimanual manipulation," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [30] K. Black *et al.*, " $\pi_{0.5}$: A vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.
- [31] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, X. Wang, B. Liu, J. Fu, J. Bao, D. Chen, Y. Shi, J. Yang, and B. Guo, "CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation," *arXiv preprint arXiv:2411.19650*, 2024.